

Лабораторная работа.

Непрерывная интеграция (dev)

🕒 Created	August 28, 2026 5:33 PM
🏷️ Tags	devops labs

1. Общие сведения о лабораторном стенде

1.1 О триггере по push и доступности Jenkins извне

Автоматический запуск сборки по событию `push` в GitHub технически реализуется через **webhook** — GitHub должен суметь достучаться до Jenkins по HTTP(S) через интернет. Если `control-node` находится в закрытой лабораторной сети без публичного IP/порта:

- используйте вариант с **Poll SCM** (раздел 3.5) — Jenkins сам с заданной периодичностью проверяет репозиторий на новые коммиты; результат для пользователя неотличим от push-триггера, только с небольшой задержкой;
- либо временно откройте туннель к `control-node` (например, `ngrok http 8080`) — исключительно для демонстрации в раздел 3.4, официально не для продуктивного использования.

2. Создание проекта с тестами

2.1 Цель раздела

Расширить демонстрационный проект `demo-app` (Python) из первой части лабораторной работы дополнительными тестами и отчётом о покрытии кода — это входные данные, которые в дальнейшем увидит Jenkins.

2.2 Структура проекта

Официальная документация pytest: <https://docs.pytest.org/en/stable/>, pytest-cov: <https://pytest-cov.readthedocs.io/>.

Перейдите в клон репозитория `demo-app` (Лабораторная работа. Системы контроля версий) и обновите `requirements.txt`:

```
pytest==8.3.3
pytest-cov==5.0.0
```

Расширьте `app.py` — добавим ещё одну функцию, чтобы было что тестировать и анализировать:

```
def add(a, b):
    return a + b

def divide(a, b):
    if b == 0:
        raise ValueError("Деление на ноль недопустимо")
    return a / b

if __name__ == "__main__":
    print(add(2, 3))
    print(divide(10, 2))
```

Дополните `test_app.py`:

```
import pytest
from app import add, divide

def test_add():
    assert add(2, 3) == 5

def test_add_negative():
    assert add(-1, -1) == -2

def test_divide():
```

```
assert divide(10, 2) == 5
```

```
def test_divide_by_zero():  
    with pytest.raises(ValueError):  
        divide(1, 0)
```

2.3 Локальный запуск тестов с отчётом о покрытии

```
python3 -m venv venv  
./venv/bin/pip install -r requirements.txt  
./venv/bin/pytest --junitxml=result.xml --cov=. --cov-report  
t=xml:coverage.xml --cov-report=term
```

Флаг `--cov-report=xml` создаёт файл `coverage.xml` в формате Cobertura, а `--junitxml` — отчёт, который умеет показывать Jenkins (раздел 5).

2.4 Зафиксируйте изменения и отправьте в GitHub

```
git switch -c feature/tests-and-coverage  
git add app.py test_app.py requirements.txt  
git commit -m "Добавлены дополнительные тесты и отчёт о пок  
рытии"  
git push -u origin feature/tests-and-coverage
```

Оформите и слейте Pull Request в `main` — дальше пайплайн будет реагировать именно на изменения в `main`.

3. Пайплайн CI в Jenkins: триггер на push, сборка, тесты

3.1 Цель раздела

Настроить автоматический запуск существующего Jenkins-пайплайна при появлении новых коммитов в ветке `main`, и расширить `Jenkinsfile` публикацией отчёта о покрытии.

3.2 Плагин GitHub в Jenkins

Официальная документация: <https://plugins.jenkins.io/github/> .

Убедитесь, что плагин **GitHub** установлен (обычно входит в набор suggested plugins): **Manage Jenkins** → **Plugins** → **Installed plugins**.

3.3 Учётные данные для доступа к репозиторию

Официальная документация GitHub по токенам:

<https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens> .

Если репозиторий приватный, создайте Personal Access Token (классический или fine-grained, с правом чтения репозитория) и добавьте его в Jenkins: **Manage Jenkins** → **Credentials** → **System** → **Global credentials** → **Add Credentials**, тип **Username with password** (в поле пароля — токен).

3.4 Настройка webhook на стороне GitHub (если Jenkins доступен извне)

Официальная документация: <https://docs.github.com/en/webhooks/using-webhooks/creating-webhooks> и

<https://www.jenkins.io/doc/book/pipeline/multibranch/#github-webhooks> .

1. В репозитории на GitHub откройте **Settings** → **Webhooks** → **Add webhook**.
2. **Payload URL**: <http://<адрес Jenkins>:8080/github-webhook/> (обратите внимание на завершающий слэш).
3. **Content type**: [application/json](#) .
4. Событие: **Just the push event**.
5. Сохраните — GitHub сразу отправит тестовый пинг, статус можно проверить во вкладке **Recent Deliveries**.

В настройках самой задачи Jenkins (**Configure** → **Build Triggers**) включите чекбокс **GitHub hook trigger for GITScm polling**.

3.5 Альтернатива: Poll SCM (без публичного адреса)

Официальная документация:

<https://www.jenkins.io/doc/book/pipeline/jenkinsfile/#triggers> .

Если webhook недоступен, в том же разделе **Build Triggers** включите **Poll SCM** и укажите расписание в формате cron:

```
H/5 * * * *
```

Это заставит Jenkins проверять репозиторий на новые коммиты каждые ~5 минут и запускать сборку только при их наличии — в отличие от `Build periodically`, которая запускает сборку по расписанию безусловно.

Тот же триггер можно задать прямо в `Jenkinsfile` (актуально при использовании Multibranch Pipeline):

```
triggers {
    pollSCM('H/5 * * * *')
}
```

3.6 Обновление Jenkinsfile: публикация покрытия

Дополните `Jenkinsfile` из лабораторной работы №1 — замените команду тестов на вариант с покрытием и явно укажите ветку сборки:

```
pipeline {
    agent any

    options {
        timestamps()
    }

    triggers {
        pollSCM('H/5 * * * *') // используйте, если webhook
        // недоступен; при рабочем webhook эту секцию можно убрать
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Setup') {
```

```

        steps {
            sh 'python3 -m venv venv'
            sh './venv/bin/pip install --upgrade pip'
            sh './venv/bin/pip install -r requirements.
txt'
        }
    }

    stage('Build') {
        steps {
            sh './venv/bin/python -m py_compile app.py'
        }
    }

    stage('Test') {
        steps {
            sh './venv/bin/pytest --junitxml=result.xml
--cov=. --cov-report=xml:coverage.xml --cov-report=term'
        }
    }
}

post {
    always {
        junit 'result.xml'
    }
    success {
        echo 'Сборка прошла успешно'
    }
    failure {
        echo 'Сборка завершилась с ошибкой'
    }
}
}

```

Зафиксируйте и отправьте изменения в `main` (напрямую или через Pull Request).

3.7 Проверка автоматического запуска

Внесите тривиальное изменение (например, в `README.md`), закоммитьте и отправьте в `main`:

```
git switch main
git pull origin main
echo "Проверка триггера CI" >> README.md
git add README.md
git commit -m "Проверка автоматического запуска пайплайна"
git push origin main
```

Через несколько секунд (webhook) или в пределах интервала опроса (Poll SCM) в Jenkins должна автоматически появиться новая сборка — без нажатия **Build Now**.

4. Просмотр результатов сборки и тестов в интерфейсе CI

4.1 Цель раздела

Научиться читать результаты выполненного пайплайна: статус стадий, отчёт о тестах, тренд сборок — то есть отличать «зелёную» сборку от «красной» не только по общему статусу, но и по деталям, которые к этому привели.

4.2 Стадии пайплайна и консольный вывод

На странице задачи откройте последнюю сборку → **Stage View** — визуально видно, сколько заняла каждая стадия и какая из них (если есть) завершилась ошибкой. **Console Output** этой сборки даёт полный текстовый лог, включая вывод pytest.

4.3 Отчёт о тестах

Официальная документация плагина JUnit: <https://plugins.jenkins.io/junit/>.

На странице сборки откройте **Test Result** — Jenkins показывает:

- количество пройденных/упавших/пропущенных тестов;
- время выполнения каждого теста;
- полный стектрейс для упавших тестов (без необходимости листать Console Output).

На странице самой задачи (не отдельной сборки) есть график **Test Result Trend** — изменение числа тестов и их статуса от сборки к сборке; полезно, чтобы вовремя заметить, что число тестов перестало расти или начало «мигать» (нестабильные тесты).

4.4 Анализ упавшей сборки: практика

Намеренно понизьте качество кода — добавьте в `app.py` неиспользуемую переменную и код с очевидным дублированием, закоммитьте и отправьте в `main`:

```
def unused_helper():  
    unused_variable = 42  
    return None
```

Проследите за сборкой:

1. В **Stage View** увидите, на какой стадии пайплайн остановился (скорее всего, Quality Gate, если новый код нарушает пороги).
2. В **Console Output** найдите сообщение Pipeline aborted due to quality gate failure.
3. Исправьте код (удалите неиспользуемую функцию), закоммитьте — убедитесь, что следующая сборка снова проходит целиком.